AIAC-Lab Assignment – 10.1

Name : Saketh Birru

HTNo : 2303A51403

Batch No : 06

Assignment : 10.1

Task-1

'''

Python script.

Sample Input Code:

Calculate average score of a student

Code :

```
def calc_average(marks):
total = 0
for m in marks:
total += m
average = total / len(marks)
return avrage # Typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)
```

Expected Output:

• Corrected and runnable Python code with explanations of the

fixes.

'''

#

Code :

```
def calc_average(marks):
    total = 0
    for m in marks:
```

```
        total += m
    average = total / len(marks)
    return average # Fixed the typo here
marks = [85, 90, 78, 92]
print("Average Score is ", calc_average(marks)) # Added closing parenthesis here
```

Task-2

'''

Task: Use AI to refactor Python code to follow PEP 8 style

guidelines.

Sample Input Code:

def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))

Expected Output:

• Well-formatted PEP 8-compliant Python code.

'''

Refactored code following PEP 8 style guidelines

Code :

```
def area_of_rect(length, breadth):
    """Calculate the area of a rectangle."""
    return length * breadth
print(area_of_rect(10, 20))
```

Task-3

'''

Task: Use AI to make code more readable without changing its

logic.

Sample Input Code:

def c(x,y):

return x*y/100

a=200

b=15

print(c(a,b))

Expected Output:

• Python code with descriptive variable names, inline comments, and clear formatting

'''

Refactored code with improved readability

Code :

```
def calculate_percentage(value, percentage):
    """Calculate the percentage of a given value."""
    return value * percentage / 100
amount = 200
percent = 15
print(calculate_percentage(amount, percent))
```

Task-4

'''

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
print("Welcome", students[1])
print("Welcome", students[2])
```

Expected Output:

• Modular code with reusable functions.

'''

Refactored code with reusable function

Code :

```
def welcome_students(students):
    """Print a welcome message for each student."""
```

```
    for student in students:
        print("Welcome", student)
students = ["Alice", "Bob", "Charlie"]
welcome_students(students)

# Task-5
'''
Task: Use AI to make the code run faster.
Sample Input Code:
# Find squares of numbers
nums = [i for i in range(1,1000000)]
squares = []
for n in nums:
squares.append(n**2)
print(len(squares))
Expected Output:
• Optimized code using list comprehensions or vectorized
operations.
'''
# Optimized code using list comprehension
Code :
nums = [i for i in range(1, 1000000)]
squares = [n**2 for n in nums] # Using list comprehension for faster execution
print(len(squares))

# Task-6
'''
Task Description #6 – Complexity Reduction
Task: Use AI to simplify overly complex logic.
Sample Input Code:
def grade(score):
```

```
if score >= 90:
return "A"
else:
if score >= 80:
return "B"
else:
if score >= 70:
return "C"
else:
if score >= 60:
return "D"
else:
return "F"
```

Expected Output:

• Cleaner logic using elif or dictionary mapping.

'''

Refactored code using elif for cleaner logic

Code :

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
grade_score = 85
print("Grade for score", grade_score, "is", grade(grade_score))
```